Week 5 Part 1 - Tutorial

BaseR, TidyR, and the CRANiverse

Check your understanding

Quiz

Your First Line of (R) Code

In this session we will be continuing the time-honoured tradition of using the 'Hello World' program to test our R environment.

Traditionally (well, since the 1970's, but computer programming isn't very old) this is the first line of code that new users of a programming language write.

To do so, you will be using the `print()` function. Start by typing the above in the block below and then adding `'Hello world'` (in inverted commas, I'll explain later) between the brackets. Click on run.

Write the R code required say hello world below:

Hello, World!

What happened?

If you want to read more about test programs, there's a good Wikipedia article on it here (Click to read more about 'Hello World')

Using R as a fancy calculator

The R console can be used to do arithmetic, much like a calculator. Write the R code required to add two plus two:

Function Calls

When we made the computer say "Hello, World", we called a function.

A function is simply a process that takes an input, manipulates it, and returns an output. To get a clearer idea of what this means, it is probably easier to just show you how to write a function.

Times Two Machine

Let's make a `Times Two` machine. It's going to take a number as an input, and return the value that is

twice the size.

The code below provides the skeleton of a new function. Between the two brackets, put a letter x. Between the curly brackets, type the following: `return(x * 2)` Run the code!

```
times_two <- function() {

}
```

Nothing happened!?

This is because the function is now stored in memory and is ready to do your bidding when you call it.

To call it type the following on a new line `times_two(2)`

My Maths Machine

Use the box below to make a different mathematics machine. You might change: 1. The operator (*, +, -, /) 2. The amount to change the number by (e.g. multiply by 3, 4, or 5)

```
my_machine <- function(x){

}
```

Functions can accept more than one variable as arguments. Try making our maths machine more flexible by allowing the user to decide what to multiply the number by.

```
my_machine <- function(x,y){

}
```

Datatypes

R can handle several ‘datatypes’. An object’s datatype determines how it is handled by the underlying code. Here are a few of them:

1. Logical: literally just TRUE or FALSE, where TRUE has a numerical value of 1, and FALSE has a numerical value of 0.

2. Numeric: As implied, this is numerical data. We will mostly be working with integer data (whole numbers), and double values (numbers with decimal points). There are more complex types (pun intended) but we don’t need them for now.

3. Text: Also known as character data. These are always in inverted commas and are usually used to pipe information to the console. Note that if you put numerical data inside inverted commas it will be treated as a character, not a number!

4. Factors: These are a weird hybrid of text and numerical data. They are used to code categorical variables. They are usually represented by key-value pairs, e.g. “Group1” = 0, “Group2” = 1, etc.

You can check an object’s type using the `typeof()` function, and can sometimes coerce values between datatypes using one of several `as` functions. e.g.

```
print(typeof(100))

## [1] "double"
```

```
print(as.character(100))

## [1] "100"

print(typeof(as.character(100)))

## [1] "character"
```

This doesn't always work though. Try running the text "hello, world" through the `as.numeric()` function below:

Note that the NA token indicates that no value could be returned by the function. It's good to know about these early on because they have a habit of haunting your code.

Assigning Variables

A variable stores a value to be used later on. We have already used variables a couple of times. The `x` in our times two machine was a variable, and believe it or not, our times two machine itself was also a sort of variable.

In R the standard way of assigning a value to a variable is using the assignment operator `<-`. For instance, to create a variable called `spam` that contains the value `"a can of"` you would write `spam <- "a can of"`. The variable `spam` would then store the characters that you have specified.

In principle you can name your variables anything you like. In practice, there are some standards you can follow to make your life easier and your code more readable. This is a topic in it's own right (broadly called 'linting') but here are some basics:

Variable names: 1. Can contain numbers but, 2. cannot start with a number (you'll get an error). 3. They cannot contain white space (i.e. spaces, tabs, newlines), 4. and they cannot contain some special characters that are reserved by R such as '-', '$', or '%', 5. in fact it's probably best not to use any special characters.

How can we use the principles above to make usable variable names?

```
thisisavariable <- 100
print(thisisavariable)

## [1] 100

thisisavariable2 <- 200
print(thisisavariable2)

## [1] 200
```

You'll probably agree that the above are not very readable. There are a couple of conventions that you can use to increase readability. Snake case separates words by an underscore '_' and Camel case uses capital letters to distinguish between words. It also helps to use short but informative variable names to save typing!:

```
thisVariable <- "camelCase"
print(thisVariable)

## [1] "camelCase"

this_variable_2 <- "snake_case"
print(this_variable_2)
```

```
## [1] "snake_case"
```

Try coming up with your own variable names below:

Once stored, your variable can be handled in the same way as the type of the value that is assigned to it:

```
one_hundred <- 100
print(one_hundred * 2)

## [1] 200
```

Types of Variables

In the previous section we just looked at variables of length = 1, but we can actually store many values into a variable. We can do this using one of several types of object depending on the use case.

c()

The first method to the concatenate function `c()`. This glues several values together in the order that they are assigned, e.g.

```
this_variable <- c(1,2,3,4)
print(this_variable)

## [1] 1 2 3 4
```

Once saved, you can either return one, all, or some of the values using square brackets `[]`:

```
this_variable <- c(1,2,3,4)
print(this_variable[1])

## [1] 1

print(this_variable[2])

## [1] 2

print(this_variable[1:2]) ### a colon provides a range of values

## [1] 1 2
```

The number in the square bracket is an index - and will point to the value stored in position of the number provided.

Try it yourself!

Matrices

A matrix is a 2-dimensional array of values. You create a matrix using the `matrix()` function which requires a few arguments: `x` = the values that you are storing, `nrows` = the number of rows in the matrix, `ncols` = the number of columns in the matrix (in a square matrix nrows will be the same as ncols), `byrow` = whether to fill up the matrix by rows or by columns (this will be clearer below).

```
### Filled up by columns by default
my_matrix <- matrix(c(1,2,3,4), nrow=2, ncol=2)
print(my_matrix)

##      [,1] [,2]
```

```
## [1,]    1    3
## [2,]    2    4

### Option to fill up by row
my_other_matrix <- matrix(c(1,2,3,4), nrow=2, ncol=2, byrow=TRUE)
print(my_other_matrix)

##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

Rather than indexing these with one value, you have to provide 2 - the first represents the row, the second represents the column:

```
### Filled up by columns by default
my_matrix <- matrix(c(1,2,3,4), nrow=2, ncol=2)
print(my_matrix[1,2])

## [1] 3

### Option to fill up by row
my_other_matrix <- matrix(1:4, nrow=2, ncol=2, byrow=TRUE)
print(my_other_matrix)

##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

Arrays

An array is a generalisation of matrix to any number of dimensions. So we don't melt any minds, we're just going to look at a 3D array. MRI data is in three dimensions (functional MRI data is in four - more on that later).

To make an array you provide a set of numbers (in this case 1-12 using the `1:12` shortcut), and the length of each dimension. Here we care going to have an array that has two rows, two columns, and three slices (my ad hoc name for our third dimension).

We index a 3D array using three numbers: `[rows, columns, slices]`. Leaving one of these entries empty returns all values.

```
myArray <- array(1:12, dim=c(2,2,3))

print(myArray[,,1]) # Show all rows and columns from the first slice

##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4

print(myArray[1,,]) # Show all columns and slices from the first row

##      [,1] [,2] [,3]
## [1,]    1    5    9
## [2,]    3    7   11

print(myArray[1,2,3]) #Show the value in the first row of the second column in the
third slice

## [1] 11
```

Dataframes

Perhaps the most common object you will work with in data science is the dataframe. These share characteristics with matrices and another object called a list. They are also the closest you can get to the behaviour of an Excel spreadsheet in that you can have columns with different data-types unlike with the other objects we have looked at. We're going to take a look at one in the wild using one of the data-sets that are included.

The following code will load the sleep data-set which includes 20 observations of differences in sleep following one of two soporific drugs.

```
data(sleep) # Loads a data-set
head(sleep, n = 5) # Prints the first five rows
tail(sleep, n = 4) # Prints the last four rows
names(sleep) # Prints the names of each column
dim(sleep) # Prints the number of rows and columns
print(sleep$group) # Print the values from a particular column
print(sleep[,"group"]) # Same as above
print(sleep[1,"group"]) # Print the first row of the group column
```

The code above is not run, so feel free to type it into the console or the box below.

We will come back to dataframes later!